Disciplined Nonlinear Programming

Daniel Cederberg, **William Zhang**, Parth Nobel, and Stephen Boyd

COPT presentation, Shanghai, May 14, 2026

Outline

Background

Modeling language responsibilities

Disciplined nonlinear programming

Example

Conclusions

Convex optimization

$$\begin{array}{ll} \text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq 0, \quad i = 1, \ldots, m \\ & Ax = b \end{array}$$

- variable $x \in \mathbf{R}^n$
- equality constraints are linear and $f_0, \ldots, f_m$ are convex
- why care about convexity?

Convex optimization

$$\begin{array}{ll} \text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq 0, \quad i = 1, \ldots, m \\ & Ax = b \end{array}$$

- variable $x \in \mathbf{R}^n$
- equality constraints are linear and $f_0, \ldots, f_m$ are convex
- why care about convexity?
  - **classic answer**: local optimum = global optimum
  - **modern answer**: very actionable because of reliable software

Disciplined convex programming (DCP)

- a grammar for specifying convex optimization problems [Grant et al., 2006]
- convex and concave functions are constructed by combining a set of **atoms**
- each atom has known curvature and monotonicity
- simple rules specify how atoms can be composed

Disciplined convex programming (DCP)

- a grammar for specifying convex optimization problems [Grant et al., 2006]
- convex and concave functions are constructed by combining a set of **atoms**
- each atom has known curvature and monotonicity
- simple rules specify how atoms can be composed
- a problem that follows the DCP rules is convex by construction
- and can be solved globally and efficiently (**in theory and in practice**)

Modeling languages implementing DCP

- Yalmip (MATLAB), CVX (MATLAB), CVXPY (Python), Convex.jl (Julia), CVXR (R)
- classic commercial software like AMPL and GAMS do not implement DCP

Modeling languages implementing DCP

- Yalmip (MATLAB), CVX (MATLAB), CVXPY (Python), Convex.jl (Julia), CVXR (R)
- classic commercial software like AMPL and GAMS do not implement DCP

- two main tasks of DCP-based modeling languages:
  - **analysis**: verify that a high level problem description follows the DCP rules
  - **synthesis**: build an equivalent problem in standard conic form:

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & Ax + s = b \\ & s \in K \end{array}$$

variables x and s, and problem data $A,\ b,\ c$ and convex cone K

Modeling languages implementing DCP

- Yalmip (MATLAB), CVX (MATLAB), CVXPY (Python), Convex.jl (Julia), CVXR (R)
- classic commercial software like AMPL and GAMS do not implement DCP

- two main tasks of DCP-based modeling languages:
  - **analysis**: verify that a high level problem description follows the DCP rules
  - **synthesis**: build an equivalent problem in standard conic form:

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & Ax + s = b \\ & s \in K \end{array}$$

variables x and s, and problem data $A,\ b,\ c$ and convex cone K

- no more communication between modeling language and solver

Nonlinear programming (NLP)

$$\begin{array}{ll} \text{minimize} & f(x) \\ \text{subject to} & c(x) = 0 \\ & \ell \leq x \leq u \end{array}$$

- objective and constraint functions are differentiable but not necessarily convex
- includes many NP-hard problems
- success depends on initialization, problem formulation, choice of algorithm

Theoretical properties

- NLP solvers have weak guarantees
  - unrealistic to expect global optimality
  - might fail to find a local minimizer (or even a feasible point)
- NLP solvers at best guarantee convergence to a KKT point
  - KKT points can be local minima, local maxima, or saddle points
  - solvers implement heuristics to steer away from undesirable KKT points
- convergence guarantees typically require constraint qualifications (CQs)
  - most common CQ: linear independence constraint qualification (LICQ)
  - we found LICQ to be important in practice

Theoretical properties

- NLP solvers have weak guarantees
  - unrealistic to expect global optimality
  - might fail to find a local minimizer (or even a feasible point)
- NLP solvers at best guarantee convergence to a KKT point
  - KKT points can be local minima, local maxima, or saddle points
  - solvers implement heuristics to steer away from undesirable KKT points
- convergence guarantees typically require constraint qualifications (CQs)
  - most common CQ: linear independence constraint qualification (LICQ)
  - we found LICQ to be important in practice
- NLP solvers still work very well in many cases (perhaps with a decent initial point)

Modeling languages for NLP

- YALMIP (MATLAB), JuMP (Julia), Pyomo (Python), and CasADi (C++)
- commercial software like AMPL, GAMS, AIMMS

Modeling languages for NLP

- YALMIP (MATLAB), JuMP (Julia), Pyomo (Python), and CasADi (C++)
- commercial software like AMPL, GAMS, AIMMS

- main task of NLP modeling languages:
  - provide solvers with derivatives
  - provide an initial point, if not specified
- communication between modeling language and solver in every iteration

Outline

Background

Modeling language responsibilities

Disciplined nonlinear programming

Example

Conclusions

Solver requirements

Interior-point methods for optimization all require atleast

- an initial point in the domain of the functions
- smoothness (twice differentiable) of the functions
- the derivatives (Jacobian and Hessian) of the functions

Solver requirements

Interior-point methods for optimization all require atleast

- an initial point in the domain of the functions
- smoothness (twice differentiable) of the functions
- the derivatives (Jacobian and Hessian) of the functions
- **Domain is not the same as feasibility**

When NLP modeling languages struggle: domain example

$$\text{minimize} \quad -\sum_i \log(A_i x - b_i)$$

- variable x
- solution is the analytic center of a polytope
- finding an initial point requires solving an LP
- existing modeling languages initialize x to the origin
- negative values will crash solvers before the first iteration

A solution

We can instead specify the following problem

$$\begin{array}{ll} \text{minimize} & -\sum_i \log(y_i) \\ \text{subject to} & y_i = A_i x - b_i, \text{ for all } i \in \{1, \ldots, n\} \end{array}$$

- with a new variable y and auxiliary constraint
- initializing $y = \mathbf{1}$ ensures a *point in domain*
- the constraints are handled with an infeasible-start method

What DCP modeling languages do

The problem is converted to the following conic form

$$\begin{array}{ll} \text{minimize} & -\sum_i t_i \\ \text{subject to} & (t_i, \mathbf{1}, A_i x - b_i) \in \mathcal{K}_{exp} \text{ for all } i \in \{1, \ldots, n\} \end{array}$$

- where $\mathcal{K}_{exp}$ is an *exponential cone* and t is a new variable
- initialization in the domain of K_{exp} is done easily

What DCP modeling languages do

The problem is converted to the following conic form

$$\begin{array}{ll} \text{minimize} & -\sum_i t_i \\ \text{subject to} & (t_i, \mathbf{1}, A_i x - b_i) \in \mathcal{K}_{exp} \text{ for all } i \in \{1, \ldots, n\} \end{array}$$

- where $\mathcal{K}_{exp}$ is an *exponential cone* and t is a new variable
- initialization in the domain of K_{exp} is done easily
- this is also true for **all standard convex cones**

When NLP modeling languages struggle: smooth derivatives

$$\text{minimize } \|Ax - b\|_2^2 + \lambda \|x\|_1$$

- variable x, regularization parameter $\lambda > 0$
- convex problem (lasso) so should be solved gracefully
- the solution occurs at a point of nondifferentiability
- objective is treated as a black box and “derivatives” are supplied to the solver
- ignoring nondifferentiability might cause solvers to struggle

When NLP modeling languages struggle: smooth derivatives

output from Ipopt [Wächter and Biegler, 2006] when interfacing it using Casadi:

```
This is Ipopt version 3.14.11, running with linear solver MUMPS 5.4.1.

...

iter    objective    inf_pr   inf_du lg(mu)  ||d||  lg(rg) alpha_du alpha_pr  ls
  10  1.1126107e+01 0.00e+00 1.71e+00  -1.7 1.29e+08  -7.8 1.00e+00 1.32e-23f 77
  11  1.3152054e+10 0.00e+00 1.71e+00  -1.7 4.09e+08  -8.3 1.00e+00 1.00e+00w  1
  12  3.5423855e+10 0.00e+00 1.66e+00  -1.7 1.13e+09  -8.8 1.00e+00 1.00e+00w  1
  13  1.4112259e+11 0.00e+00 1.58e+00  -1.7 3.39e+09  -9.2 1.00e+00 1.00e+00w  1
Cannot call restoration phase at point that is almost feasible (violation 0.000000e+00).
Abort in line search due to no other fall back.

Number of Iterations....: 13

EXIT: Error in step computation!
```

What DCP modeling languages do

The problem can get rid of $|x|$ by the equivalent transformation to

$$\begin{array}{ll} \text{minimize} & \|Ax - b\|_2^2 + \lambda \sum_i t_i \\ \text{subject to} & -t \leq x \leq t \end{array}$$

- where t is a new variable
- first move $|x|$ to the constraints as $|x| \leq t$
- equivalent because of monotonicity
- all functions are now twice differentiable

How NLP modeling languages compute derivatives

- automatic differentiation of the provided functions
- sparsity is handled through graph coloring
- hessian is often expensive and difficult to compute
- an approximation can be used instead (L-BFGS)

How NLP modeling languages compute derivatives

- automatic differentiation of the provided functions
- sparsity is handled through graph coloring
- hessian is often expensive and difficult to compute
- an approximation can be used instead (L-BFGS)
- can still be the **bottleneck** for certain problems

What DCP modeling languages do

- Nothing.

What DCP modeling languages do

- Nothing.
- the problem data A, b, c of the conic form are already the derivatives of interest
- can insert them directly in the Hessian of the KKT matrix
- all standard convex cones have barrier functions with **analytic derivatives**

Who is responsible?

- the solver

Who is responsible?

- the solver
- no, it's the user

Who is responsible?

- the solver
- no, it's the user
- also no, it should be the modeling language

Outline

Background

Modeling language responsibilities

Disciplined nonlinear programming

Example

Conclusions

Disciplined nonlinear programming (DNLP)

- a new grammar for specifying nonlinear programs
- smooth functions can be freely mixed with nonsmooth functions
- rules governing how nonsmooth functions can appear
- smooth in the interior of their domain

Disciplined nonlinear programming (DNLP)

- a new grammar for specifying nonlinear programs
- smooth functions can be freely mixed with nonsmooth functions
- rules governing how nonsmooth functions can appear
- smooth in the interior of their domain

a DNLP-compliant problem can be transformed to a form that

- only involves smooth functions
- does not introduce LICQ violations in the transformation

Atoms

in DNLP, users represent functions as **expressions** built from **atoms** with known attributes

- atoms are divided into three categories:
  - **smooth**
  - **nonsmooth convex**
  - **nonsmooth concave**
- atoms have known monotonicity in each of its arguments (increasing or decreasing)

A few atoms

Atom	Definition	Domain
	Smooth, nonconvex and nonconcave	
`multiply`	$\phi(x, y) = xy$	
`quad_form`	$\phi(x) = x^T Qx$ where $Q \in \mathbf{S}^n$	
`sin`	$\phi(x) = \sin x$	
`sigmoid`	$\phi(x) = \frac{1}{1+e^{-x}}$	
	Smooth, convex or concave	
`log`	$\phi(x) = \log x$	$x > 0$
`sqrt`	$\phi(x) = \sqrt{x}$	$x \geq 0$
	Nonsmooth, convex	
`abs`	$\phi(x) = \lvert x \rvert$	
`max`	$\phi(x) = \max\{x_1, x_2, \ldots, x_n\}$	
`norm1`	$\phi(x) = \lVert x \rVert_1$	
	Nonsmooth, concave	
`min`	$\phi(x) = \min\{x_1, x_2, \ldots, x_n\}$	
`sum_smallest`	$\phi(x; k) = \sum_{i=n-k+1}^{k} x_{[i]}$ where $k \in \{1, \ldots, n\}$	

Expressions

expressions (and subexpressions) are classified as

- **smooth**
- **linearizable-convex** (L-convex)
- **linearizable-concave** (L-concave)

Classification of expressions

an expression $f(x) = \phi(g(x))$ (with $\phi : \mathbf{R} \to \mathbf{R}$ an atom and $g : \mathbf{R}^n \to \mathbf{R}$ a subexpression)

- is **smooth** if ϕ is a smooth atom and $g(x)$ is a smooth expression
- is **L-convex** if ϕ is a smooth or nonsmooth convex atom and
  - $g(x)$ is a smooth expression, or
  - $g(x)$ is L-convex and ϕ is increasing, or
  - $g(x)$ is L-concave and ϕ is decreasing
- **example:** $f(x) = |x^T Q x + c|$ represented by `abs(quad_form(x, Q) + c)` is L-convex

similar definition for L-concave expressions and expressions with $g : \mathbf{R}^n \to \mathbf{R}^m$

Objectives and constraints

a problem is a **disciplined nonlinear program** (DNLP problem) if:

- **objective**: minimize an L-convex expression (or maximize an L-concave expression)
- **equality constraints**: both left and right-hand sides are smooth expressions
- **inequality constraints ($\leq$)**: left-hand side is L-convex and right-hand side is L-concave

Connection to DCP

DNLP is closely related to convex optimization and DCP

- a DNLP problem is one that is DCP if all its smooth atoms are treated as affine
- an L-convex expression is one that is convex if all its smooth atoms are linearized (or treated as affine)

Dealing with limited domain atoms

We take the same approach as the analytic center example

- the expression $\phi(Ax - b)$ becomes $\phi(y)$ with a new constraint
- we always ensure that y is initialized in the domain of ϕ
- for nonsmooth atoms, we also initialize the epigraph variable to something reasonable
- variables with bounds are also always satisfied

Dealing with nonsmooth atoms

- a DNLP problem is equivalent to a problem where any
  - nonsmooth convex atom ϕ appears in a constraint of the form $\phi(x) \leq t$
  - nonsmooth concave atom ϕ appears in a constraint of the form $\phi(x) \geq t$

Dealing with nonsmooth atoms

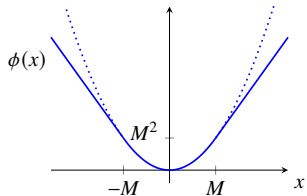
- a DNLP problem is equivalent to a problem where any
  - nonsmooth convex atom ϕ appears in a constraint of the form $\phi(x) \leq t$
  - nonsmooth concave atom ϕ appears in a constraint of the form $\phi(x) \geq t$
- these constraints can be formulated using smooth functions [Grant and Boyd, 2008]
  - the reformulations do not introduce LICQ violations
- $|x| \leq t$ can be reformulated as $-t \leq x \leq t$

Dealing with nonsmooth atoms

- a DNLP problem is equivalent to a problem where any
  - nonsmooth convex atom ϕ appears in a constraint of the form $\phi(x) \leq t$
  - nonsmooth concave atom ϕ appears in a constraint of the form $\phi(x) \geq t$
- these constraints can be formulated using smooth functions [Grant and Boyd, 2008]
  - the reformulations do not introduce LICQ violations
- $|x| \leq t$ can be reformulated as $-t \leq x \leq t$
- other nonsmooth atoms, such as the Huber atom

$$\phi(x) = \begin{cases} x^2 & |x| \leq M \\ 2M|x| - M^2 & |x| > M \end{cases}$$

have more complex reformulations



Our approach to computing derivatives

`https://github.com/SparseDifferentiation/SparseDiffPy`

- initially used CVXPY's `grad` functionality
- developed a C package for sparse differentiation with Python bindings
- every atom implements its own forward, jacobian and hessian
- our goal is still to not be the bottleneck (spend less time than in the solver)

Outline

Background

Modeling language responsibilities

Disciplined nonlinear programming

Example

Conclusions

A DNLP extension of CVXPY

`https://github.com/cvxgrp/DNLP`

- extends CVXPY with DNLP functionality
- supported solvers:
  - IPOPT (open-source, interior-point method)
  - Uno [Vanaret and Leyffer, 2025] (open-source, IPM and SQP)
  - KNITRO (commercial, many algorithms)
  - COPT
- integrated in CVXPY and will be released in v1.9

Circle packing

$$\begin{array}{ll} \text{minimize} & \max_{i=1,\ldots,n} \left(\|c_i\|_\infty + r_i\right) \\ \text{subject to} & \|c_i - c_j\|_2^2 \geq (r_i + r_j)^2, \quad 1 \leq i < j \leq n, \end{array}$$

- goal is to arrange n circles with given radii r_i into smallest possible square
- variables are the circle centers $c_i \in \mathbf{R}^2$
- objective is L-convex, constraint functions are smooth

Circle packing

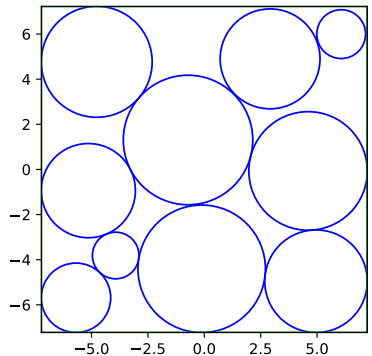
$$\begin{array}{ll} \text{minimize} & \max_{i=1,\ldots,n} \left(\|c_i\|_\infty + r_i\right) \\ \text{subject to} & \|c_i - c_j\|_2^2 \geq (r_i + r_j)^2, \quad 1 \leq i < j \leq n, \end{array}$$

- goal is to arrange n circles with given radii r_i into smallest possible square
- variables are the circle centers $c_i \in \mathbf{R}^2$
- objective is L-convex, constraint functions are smooth

```
c, constr = Variable((n, 2)), []
for i in range(n - 1):
    constr += [sum((c[i, :] - c[i+1:, :]) ** 2, axis=1) >= (r[i] + r[i+1:]) ** 2]
cost = max(norm_inf(c, axis=1) + r)
prob = Problem(Minimize(cost), constr)
c.value = uniform(-5.0, 5.0, (n, 2))            # random initial point
prob.solve(nlp=True)                            # or prob.solve(nlp=True, best_of=500)
```

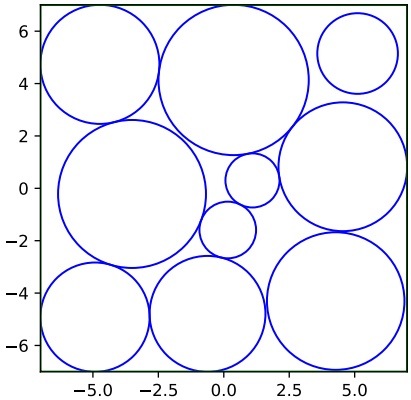
Circle packing

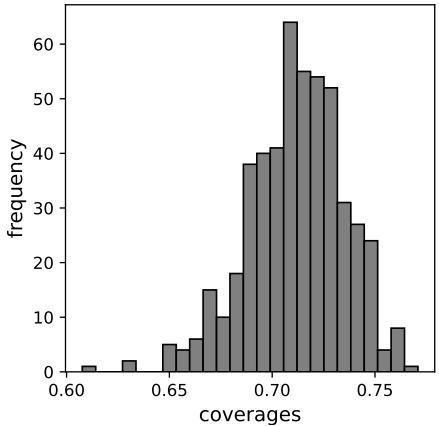
- result for $n = 10$ circles with random radii and random initial point
- the fraction of the square covered by the circles is 0.72



Circle packing

- result for same problem instance over 500 random initializations using `best_of=500`
- best coverage found is 0.77





More examples

`https://github.com/cvxgrp/dnlp-examples`

- nonlinear optimal control of a car
- portfolio construction and options pricing
- optimal power flow and state estimation
- path planning with obstacle avoidance

Outline

Background

Modeling language responsibilities

Disciplined nonlinear programming

Example

Conclusions

Conclusions

- DNLP is a new grammar for specifying nonlinear programs
  - no rules if all functions are smooth
  - mitigates solver failures caused by nonsmoothness
- our DNLP-extension of CVXPY makes it easy to use NLP solvers
  - enables high level, human readable problem descriptions
  - automatically generates smooth reformulations of nonsmooth atoms
  - ideal for teaching and research
- we welcome ideas for applications and collaborations